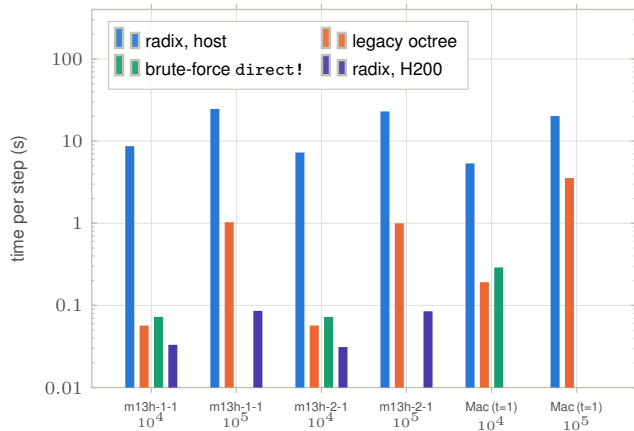
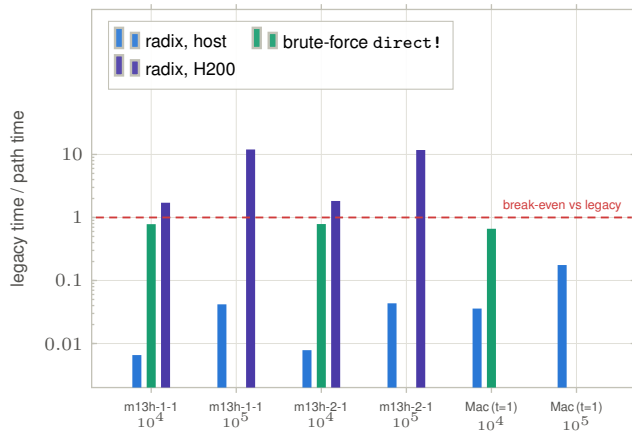


Fig. 8 — The end-user comparison: the radix path against the shipping legacy octree f_{mm} !

(a) per-step wall time



(b) speedup over the legacy octree



Machines / regime: m13h-1-1 and m13h-2-1 (Julia 1.11.7, threads=8, NVIDIA H200) and a local Apple-silicon Mac (Julia 1.12.5, threads=1, no GPU); $P = 4$, $\ell = 4$, $n \in \{10^4, 10^5\}$. Thread counts come from the run sidcar .md files — the CSVs omit them. **Data:** task 023, production_integration/benchmark_023_{m13h-1-1,m13h-2-1,mecsr-MacBook-Pro-188.local}_*.csv (tables fig08a_*, fig08b_*). **Reads as:** the *device-resident* radix path is a real end-user win at $P = 4$ — $1.71\times$ the legacy octree at $n = 10^4$ growing to $11.99\times$ at $n = 10^5$ on m13h-1-1 ($1.83\times / 11.76\times$ on m13h-2-1). The *host* radix path is not: it is $152\times$ slower than legacy at $n = 10^4$ and $24\times$ slower at $n = 10^5$, and at $n = 10^4$ it is $\approx 120\times$ slower than brute-force direct! — i.e. slower than doing no FMM at all. That is consistent with the 019b decision: the legacy octree remains the CPU default and the radix cache exists to feed the GPU loop. **There is no roadmap item to optimize the host radix path.** **Caveats — this is not an apples-to-apples comparison, in four independent ways, none of them disclosed in task 023.** (1) *Threading.* src/fmm.jl has nine Threads.@threads/@spawn sites; the radix host path (src/translate_batched{,_resident}.jl) has none. Legacy is multithreaded and host radix is not, which is why the gap collapses from $152\times$ at 8 threads to $28\times$ at 1 thread on the Mac. BLAS threads were never set or recorded. (2) *Different admissibility criteria.* Radix uses ConstantPAnalyticStencil at stencil_epsilon=1e-4 on a fixed uniform 2^4 grid; legacy uses multipole_acceptance=0.4 with leaf_size=20 on a shrunk adaptive octree. Same P , structurally different work. (3) *No error check at the benchmarked n .* The 023 script computes no error norms and never uses the direct! result as a reference; the only accuracy evidence in task 023 is at $n = 500$. (4) *Asymmetric timing boundary, against legacy.* The legacy timing includes both octree builds and the interaction-list build on every call (src/fmm.jl:1037-1058), while the radix *step* excludes its one-shot cache construction, recorded separately in Fig. 3(c) — so the boundary favours radix and legacy still wins by $152\times$; the bias understates how slow the host path is. **Protocol:** times are the *minimum* over repetitions (radix 5 jittered steps, legacy 3 reps); direct! is a *single* un-repeated @elapsed and was run only at $n = 10^4$. Missing bars are unrun configurations, not zeros. **Scope:** every bar is a complete evaluation — far field *and* nearfield — on both paths.